

LPDDR4动态调频方案

1.问题背景

产品板在高温下需要降低频率才能够稳定运行,目前打算采用监控温度sensor,当温度较高时降低DDR工作频率的方式提高稳定。

该方案依赖于软件通过进程调度的方式实现。

2.实现方案

2.1 调频挡位

LPDDR4调频的情况分为两种,一种是同一挡位内进行调频,一种是不同挡位的调频。

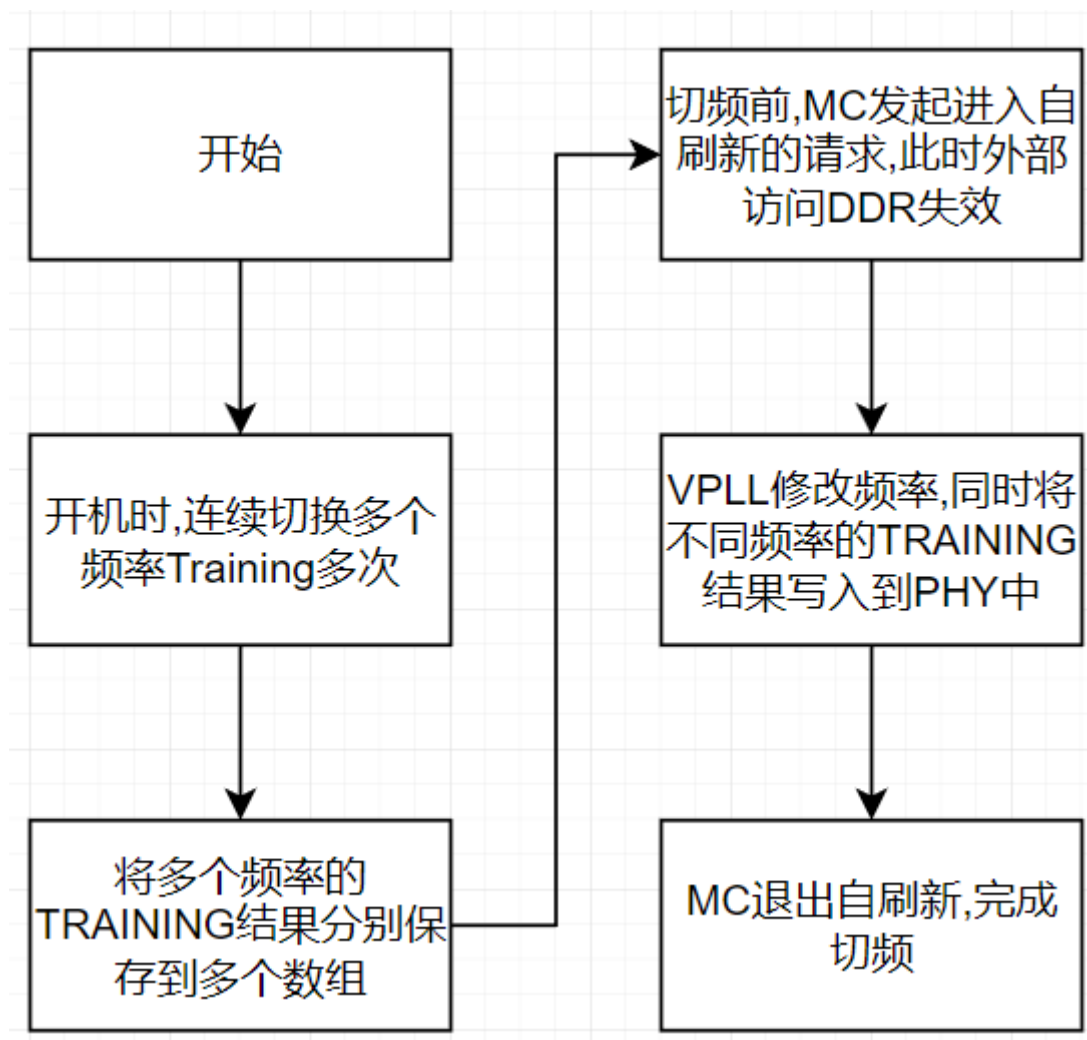
- a. 1866~2133属于同一挡位,在该挡位内的调频,只需要修改DDR PHY Training的delay line结果,即可实现切频的目的
- b. 1066~1333与1866~2133属于不同挡位,在这两个挡位进行切频的时,需要额外通过JEDEC规定的调频协议

Table 102 — WDQS_on / WDQS_off Definition

WL		nWR	nRTP	WDQS_on (max)		WDQS_off (min)		Lower Clock Frequency Limit (>)	Upper Clock Frequency Limit (≤)
Set A	Set B			Set A	Set B	Set A	Set B		
4	4	6	8	0	0	15	15	10	266
6	8	10	8	0	0	18	20	266	533
8	12	16	8	0	6	21	25	533	800
10	18	20	8	4	12	24	32	800	1066
12	22	24	10	4	14	27	37	1066	1333
14	26	30	12	6	18	30	42	1333	1600
16	30	34	14	6	20	33	47	1600	1866
18	34	40	16	8	24	36	52	1866	2133
nCK	nCK	nCK	nCK	nCK	nCK	nCK	nCK	MHz	MHz
NOTE 1 WDQS_on/off requirement can be ignored when WDQS_on/off timing is overlapped with read operation period including Read burst period and tRPST or overlapped with turn-around time (RD-WT or WT-RD).									
NOTE 2 The period during which DQS is toggling because of a Read or Write can be counted as part of the WDQS_on/off requirement.									

2.1.1 同挡位调频

目前高温切频只需要满足同挡位的切频即可。高温切频的工作流程如下图所示:



2.1.2 不同挡位切频

2.1.2.1 MC:

控制器中存储有F0/F1/F2三套参数,其中F0仅在BOOT阶段使用,实际WORK阶段使用的参数仅有F1/F2,切换F1/F2依赖于MC提供的LPC硬件接口,通过该接口提供的协议,进行LPDDR的切频

Table 10-7. LPC Hardware Interface Signals

Signal	Direction	Description
<u>lp_ext_ack</u>	Output	Hardware interface acknowledge of the external request. This signal indicates that the hardware interface may issue low power commands. Once granted, all other low power command sources are disabled.

Signal	Direction	Description
<u>lp_ext_cmd</u>	Input	Defines the low power command requested through the hardware interface. This signal is only valid when the <u>lp_ext_cmd_strb</u> signal is pulsed for 1 controller clock, and must be stable when the strobe is asserted. The bits are defined as follows: • Bits [5:4] = Clock Gating. When bits [1:0] are 'b10, the only supported value is 'b00. When bits [1:0] are 'b00 or 'b01, supported settings are: – 'b00 = No clock gating – 'b01 = Memory clock gating – 'b10 = Memory and controller clock gating • Bits [3:2] = Low Power State. When bits [1:0] are 'b10, the only supported value is 'b00. When bits [1:0] are 'b00 or 'b01, supported settings are: – 'b00 = No low power state; used only for low power exit – 'b01 = Self-Refresh Power-Down Short – 'b10 = Self-Refresh Power-Down Long • Bits [1:0] = Entry/Exit/DFS. Supported settings are: – 'b00 = DFS operation – 'b01 = Low power state entry – 'b10 = Low power state exit Note: Not all combinations of these bits are supported. Refer to Table 10-6, "Supported Hardware Interface Commands" for the specific commands that are valid. Note that if an invalid command is issued on the <u>lp_ext_cmd</u> signal, unpredictable behavior will occur including the possibility that the low power control module could hang.
<u>lp_ext_cmd_strb</u>	Input	Once the external request is received and acknowledged, this signal is used to indicate a valid request on the <u>lp_ext_cmd</u> signal. This signal should be pulsed for one controller clock, and only when both the <u>lp_ext_req</u> and <u>lp_ext_ack</u> signals are asserted to 'b1. If the memory is already in a low power state, only a command that specifies a deeper low power state will be executed.
<u>lp_ext_done</u>	Output	LPC module indication that the last command from the hardware interface has been completed. This signal will be cleared to 'b0 when a command is accepted and remain at 'b0 until the command has completed. Once asserted to 'b1, this signal will remain at 'b1 until the next command is received. When this signal is cleared, any writes to the <u>lp_ext_cmd</u> signal will result in unpredictable behavior.
<u>lp_ext_fc_reg_copy</u>	Input	Defines which frequency-dependent timing parameters and mode register parameters to use.
<u>lp_ext_req</u>	Input	External request to manage the LPC module. During the frequency scaling protocol, the <u>lp_ext_req</u> must be asserted and held until the entire procedure has completed,. Once de-asserted, this signal should not be re-asserted until the <u>lp_ext_ack</u> signal is cleared to 'b0.
<u>lp_ext_resp</u>	Output	• Bit [1] = Reserved • Bit [0] = Defines if the command was valid. Or, for DFS commands, defines the response to the last frequency scaling command from the hardware interface. This signal is only valid on the cycle that the <u>lp_ext_done</u> signal is asserted to 'b1 following either of the DFS commands. A value of "0" on this signal indicates that the command completed without errors. A value of "1" indicates that an error occurred.

2.1.2.1.1 通过LPC硬件接口进行切频的步骤

1. 在请求更改硬件频率之前，必须对这些参数进行编程,以下参数指示是切换到F1还是F2

a. FSP控制和状态参数

- fsp0_frc:标识控制器中的哪个频率集（fN）与内存设备中的频率集0相匹配。
- fsp0_frc_valid:定义fsp0_frc参数中的值是否有效。一旦用户在fsp0_frc参数中编程了一个有效值，该参数也必须设置为'b1

- fsp1_frc:标识控制器中的哪个频率集（fN）与内存设备中的频率集1相匹配。
- fsp1_frc_valid:定义fsp1_frc参数中的值是否有效。一旦用户在fsp1_frc参数中编程了一个有效值，该参数也必须设置为'b1
- fsp_wr_current:标识将在模式寄存器写入命令期间写入到具有双频设置点的模式寄存器的FSP集。用户可能正在使用一个频率设置点，并且想要更新另一个，或者更新当前正在使用的频率设置点。这应该与PHY/PI模块中引用的集合和存储器模式寄存器MR13[6]中定义的值相匹配
- fsp_op_current:标识当前正在使用的FSP集。这应该与PHY/PI模块中引用的集合和存储器模式寄存器MR13[7]中定义的值相匹配
- mr_fsp_data_valid_f0:报告频率集N的内存是否经过训练，数据是否加载到控制器的mrX_data_fN参数中。一旦设置了一个位，这些值就可以用于频率变化。除非在“频率变化退出”期间lp_ext_fc_reg_copy信号被驱动为非零，否则将使用频率设置0。
- mr_fsp_data_valid_f1:报告频率集N的内存是否经过训练，数据是否加载到控制器的mrX_data_fN参数中。一旦设置了一个位，这些值就可以用于频率变化。除非在“频率变化退出”期间lp_ext_fc_reg_copy信号被驱动为非零，否则将使用频率设置0。
- F2->F1的参考配置:

○ DDR定时和控制参数

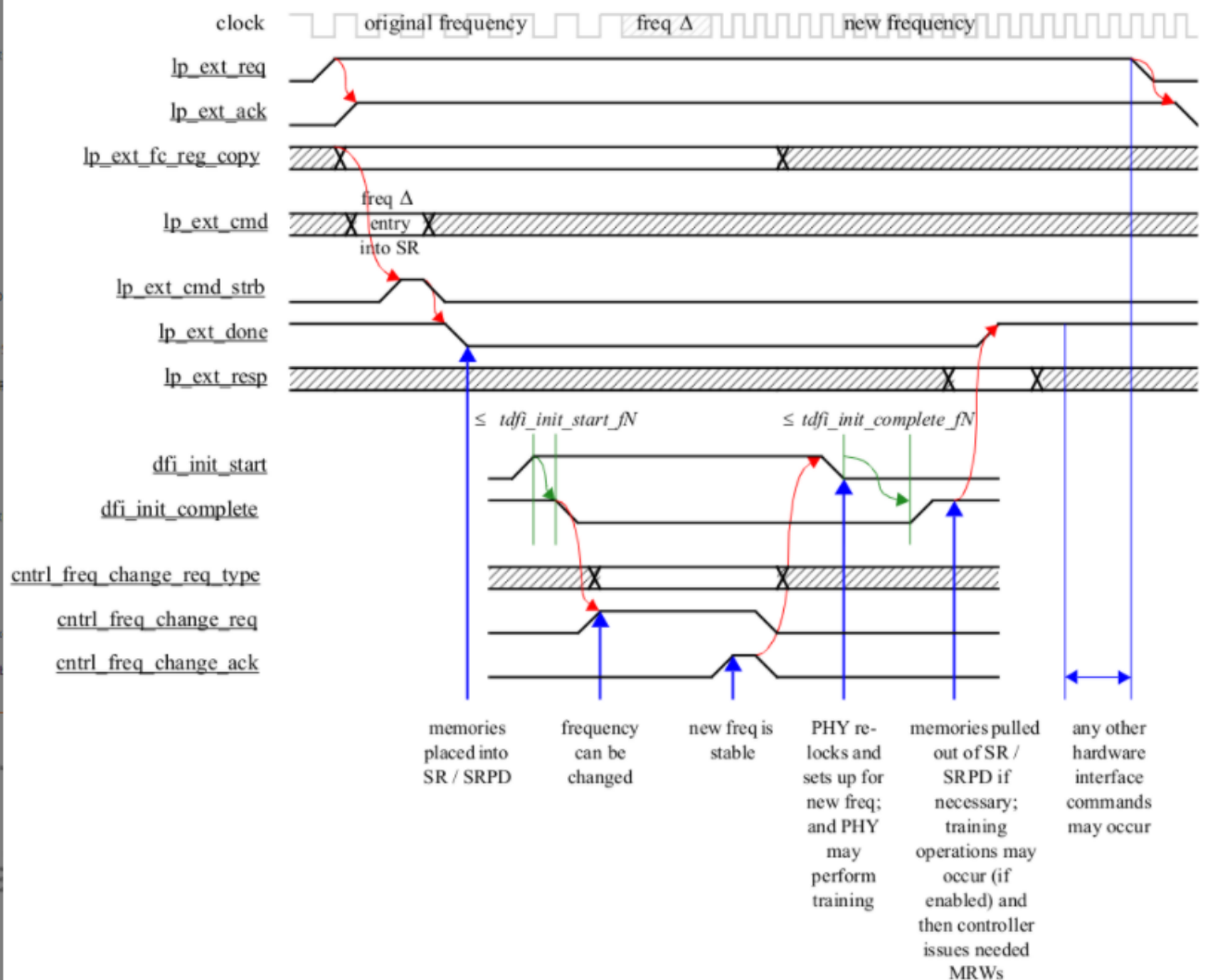
```

115 void fsp_param_conf(void){
116     uint32_t reg_value = 0;
117     reg_value = (read_register(0x0000037c)&(~(0x3<<24))|(0<<24);
118     write_register(0x0000037c,reg_value);//fsp0_frc
119
120     reg_value = (read_register(0x0000037c)&(~(0x1<<8))|(1<<8);
121     write_register(0x0000037c,reg_value);//fsp0_frc_valid
122
123     reg_value = (read_register(0x00000380)&(~(0x3))|(1<<0);
124     write_register(0x00000380,reg_value);//fsp1_frc
125
126     reg_value = (read_register(0x0000037c)&(~(0x1<<16))|(1<<16);
127     write_register(0x0000037c,reg_value);//fsp1_frc_valid
128
129     reg_value = (read_register(0x0000037c)&(~(0x1))|(1<<0);
130     write_register(0x0000037c,reg_value);//fsp_wr_current
131
132     reg_value = (read_register(0x00000378)&(~(0x1<<24))|(0<<24);
133     write_register(0x00000378,reg_value);//fsp_op_current
134
135     reg_value = (read_register(0x00000370)&(~(0x1<<24))|(1<<24);
136     write_register(0x00000370,reg_value);//mr_fsp_data_valid_f0
137
138     reg_value = (read_register(0x00000374)&(~(0x1))|(1<<0);
139     write_register(0x00000374,reg_value);//mr_fsp_data_valid_f1
140 }
141

```

ii. 完成dfs的参数配置之后,需要通过配置寄存器的方式,完成LPC的调频协议

Figure 10-4. Frequency Change Timing Diagram



调频协议标准如上图所示

1. 设置频率相关的定时参数和模式寄存器参数的集合
 - a. FSP控制和状态参数
 - b. DDR定时和控制参数
 - c. MR寄存器参数
 - d. PHY控制参数
 - e. FAM特定参数赋值
2. 启用lpc硬件接口
 - a. 断言lp_ext_req信号，成功后可发送调频(DFS)/休眠/唤醒等命令
 - b. 保持断言的lp_ext_req信号，直到DFS命令生效
 - c. 等待lp_ext_ack信号为1，硬件接口获得控制权，此时MC可接受DFS调频请求
3. 发送DFS请求
 - a. 配置lp_ext_cmd = 0x4(4'b0100)

b. 拉高lp_ext_cmd_strb信号一个控制器时钟，使lp_ext_cmd发送

c. DDR颗粒行为

i. DDR进入SR/SRPD

ii. 频率改变

iii. 新频率稳定

iv. PHY为新频率重新定位和设置

v. 从SR/SRPD中退出；训练操作可能发生（如果启用），然后控制器发出所需的MRW

4. 判断lp_ext_done信号拉低，则DDR完成DFS

5. 结束DFS，检查lp_ext_done是否为1,检查lp_ext_resp[0]信号位上的值是否为0,硬件接口将把存储器置于DFS命令指定的低功率状态

6. 拉低lp_ext_req

调频参考代码

```
142 void enter_dfs(void){
143     uint32_t reg_value = 0;
144     uint32_t cntrl_freq_change_req_type = 0;
145     uint32_t lp_ext_ack = 0;
146     uint32_t lp_ext_done = 1;
147     uint32_t lp_ext_resp = 1;
148
149     reg_value = (read_apbmux_register(0x00000008)&(~(0x3f)))|(0x4);
150     write_apbmux_register(0x00000008,reg_value);//set lp_ext_cmd
151     reg_value = (read_apbmux_register(0x00000008)&(~(3<<7))|(1<<7);
152     write_apbmux_register(0x00000008,reg_value);//assert lp_fc_reg_copy
153     reg_value = (read_apbmux_register(0x00000008)&(~(1<<9))|(1<<9);
154     write_apbmux_register(0x00000008,reg_value);//assert lp_ext_req
155     while(!lp_ext_ack){
156         lp_ext_ack = read_apbmux_register(0x00000010)&(0x1);
157         printf("!lp_ext_ack\n");
158     }
159     reg_value = read_apbmux_register(0x00000008)&(~(0x1<<6));
160     write_apbmux_register(0x00000008,reg_value|(0x1<<6));//lp_ext_cmd_strb to 1
161     write_apbmux_register(0x00000008,reg_value);//lp_ext_cmd_strb to 0
162     while(lp_ext_done){
163         lp_ext_done = (read_apbmux_register(0x00000010)>>1)&0x1;
164         printf("lp_ext_done\n");
165     }
166     printf("start dfs\n");
167     while(1){
168         cntrl_freq_change_req_type = (read_apbmux_register(0x00000014)>>23)&0x3;
169         if(cntrl_freq_change_req_type==0){
170             writel(29, 0xc2f08000 + 0xc0 + 0x74); // set ratio, mpu_clk=mpu_clk_x2/2
171             lpddr4_clksrc_select(1);
172             reg_value = read_apbmux_register(0x00000004)&(~0x1);
173             write_apbmux_register(0x00000004,reg_value|0x1);//cntrl_freq_change_req_ack to 1
174             write_apbmux_register(0x00000004,reg_value);//cntrl_freq_change_req_ack to 0
175             break;
176         }else if(cntrl_freq_change_req_type==1){
177             lpddr4_pll_freq(533);
178             lpddr4_clksrc_select(3);
179             reg_value = read_apbmux_register(0x00000004)&(~0x1);
180             write_apbmux_register(0x00000004,reg_value|0x1);//cntrl_freq_change_req_ack to 1
181             write_apbmux_register(0x00000004,reg_value);//cntrl_freq_change_req_ack to 0
182             break;
183         }
```

```

183         }else if(cntnl_freq_change_req_type==2){
184             lpddr4_pll_freq(2133);
185             lpddr4_clksrc_select(3);
186             udelay(1);
187             reg_value = read_apbmux_register(0x00000004)&(~0x1);
188             write_apbmux_register(0x00000004,reg_value|0x1);//cntnl_freq_change_req_ack to 1
189             write_apbmux_register(0x00000004,reg_value);//cntnl_freq_change_req_ack to 0
190             break;
191         }
192     }
193
194     while(!lp_ext_done){
195         lp_ext_done = (read_apbmux_register(0x00000010)>>1)&0x1;
196     }
197     printf("dfs done\n");
198     lp_ext_resp = (read_apbmux_register(0x00000010)>>2)&0x1;
199     if(lp_ext_resp){
200         printf("dfs error\n");
201     }
202     reg_value = read_apbmux_register(0x00000008)&(~(1<<9));
203     write_apbmux_register(0x00000008,reg_value);//deassert lp_ext_req
204 }
205

```

2.1.2.2 PHY

PHY在MC完成DFS后,需要写入F1频率的TRAINING结果,TRAINING结果的写入可参考

[DDR Loopback调试指南](#)

2.1.2.3 DDR颗粒

DDR颗粒与MC类似,内部存有两套MR寄存器,当发出切频请求时,DDR颗粒完成两套MR寄存器的切换

理论上MC通过LPC硬件接口完成调频的时候,已经满足DDR颗粒的调频协议,以下内容仅供参考

- 以下MR寄存器具有两套:

- mr1_data_fN_X
- mr2_data_fN_X
- mr3_data_fN_X
- mr11_data_fN_X
- mr12_data_fN_X
- mr14_data_fN_X
- mr22_data_fN_X

- DDR颗粒的切频协议:

- 1.DDR的切频协议通过MR13实现

Table 140 — Relation between MR Setting and DRAM Operation

Function	MR# & Operand	Data	Operation	Note
FSP-WR	MR13 OP[6]	0 (Default)	Data write to Mode Register N for FSP-OP[0] by MRW Command. Data read from Mode Register N for FSP-OP[0] by MRR Command.	1
		1	Data write to Mode Register N for FSP-OP[1] by MRW Command. Data read from Mode Register N for FSP-OP[1] by MRR Command.	
FSP-OP	MR13 OP[7]	0 (Default)	DRAM operates with Mode Register N for FSP-OP[0] setting.	2
		1	DRAM operates with Mode Register N for FSP-OP[1] setting.	

NOTE 1 FSP-WR stands for Frequency Set Point Write/Read.
NOTE 2 FSP-OP stands for Frequency Set Point Operating Point.

2.DDR颗粒的切频协议

Figure 119: Example of Switching Between Two Trained Frequency Set Points

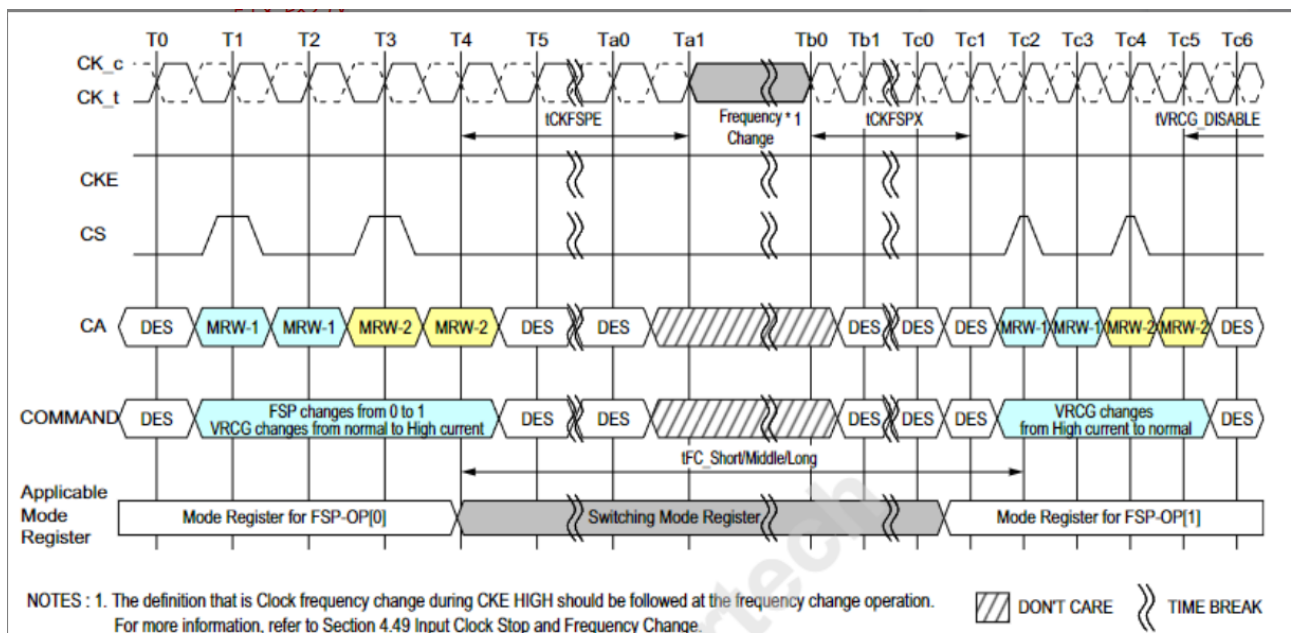
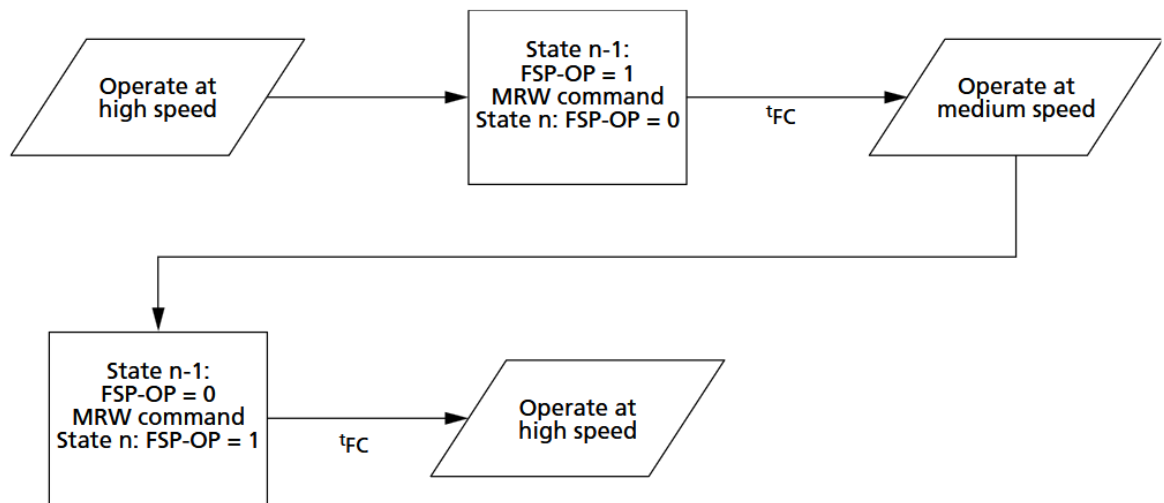


Figure 118 — Frequency Set Point Switching Timing

MR13 将FSP-OP写为1发起调频请求,同时DDR颗粒进入高电流模式,经过tFC的事件后,DDR颗粒完成调频请求,成果将MR寄存器切换到另一组

两组MR寄存器的写入均是在CA TRAINING阶段完成的